

Palindrome Checker Function

Detailed Explanation

Function Purpose
The isPalindrome function aims to determine if a given string is a palindrome. A palindrome reads the same forwards and backwards, ignoring spaces, punctuation, and capitalization.

Steps to Achieve This

1. Normalize the String:

Remove Non-Alphanumeric Characters:

Use preg_replace with the pattern '/[^a-zA-Z0-9]/' to strip out any characters that are not letters or digits.

This ensures that spaces, punctuation, and other special characters do not affect the palindrome check.

Convert to Lowercase

Use strtolower to convert the entire string to lowercase. This makes the check case-insensitive, treating 'A' and 'a' as equivalent.

2.Check for Palindrome:

Reverse the String:
Use strrev to reverse the normalized string.

Compare Original and Reversed Strings:
If the normalized string is the same as the reversed string, then the input is a palindrome.

Time Complexity

The time complexity of this function is $O(n)$, where n is the length of the input string. This complexity arises from:

Normalization: The preg_replace function iterates over the input string to remove non-alphanumeric characters, which takes $O(n)$ time.

Lowercase Conversion: The strtolower function also processes each character once, contributing $O(n)$ time.

Reversal: The strrev function creates a reversed copy of the normalized string, which is $O(n)$ in time complexity.

Comparison: The final comparison of the normalized and reversed strings is $O(n)$.

```
/**
 * Check if a given string is a palindrome.
 *
 * This function normalizes the input string by removing all spaces, punctuation,
 * and converting all characters to lowercase. It then checks if the normalized
 * string reads the same forwards and backwards.
 *
 * @param string $input The string to check.
 * @return bool True if the string is a palindrome, false otherwise.
 */
function isPalindrome(string $input): bool
{
    // Normalize the string: remove non-alphanumeric characters and convert to lowercase
    $normalizedString = preg_replace('/[^a-zA-Z0-9]/', '', $input);
    $normalizedString = strtolower($normalizedString);

    // Check if the normalized string is the same forwards and backwards
    $reversedString = strrev($normalizedString);

    return $normalizedString === $reversedString;
}

// Example usage
$inputString = "A man, a plan, a canal, Panama!";
if (isPalindrome($inputString)) {
    echo "'$inputString' is a palindrome.";
} else {
    echo "'$inputString' is not a palindrome.";
}
```

Example Usage

```
$inputString = "A man, a plan, a canal, Panama!";
if (isPalindrome($inputString)) {
    echo "'$inputString' is a palindrome.";
} else {
    echo "'$inputString' is not a palindrome.";
}
```

In this example:

The input string "A man, a plan, a canal, Panama!" is normalized to "amanaplanacanalpanama".
The normalized string is reversed to "amanaplanacanalpanama".
Since the normalized string is the same forwards and backwards, the function correctly identifies it as a palindrome.

Space Complexity

The space complexity is also $O(n)$ because:

The normalized string is stored in memory.

The reversed string is another copy stored in memory.